

2 Wheel Balance Robot

ENCE 3231

Professor Goncalo Martins

06.05.2026

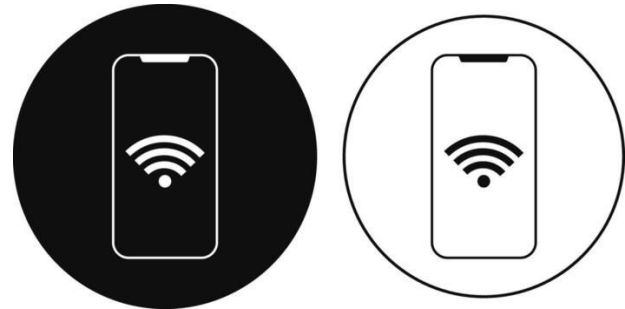
By: Teeb Alroshood

Introduction:

This project focuses on the design, development, and implementation of a two/three-wheel robot that can be remotely controlled using a smartphone. The primary objective is to apply principles of mechanical design, embedded systems, and wireless communication to create a functional mobile robot.

The robot chassis will be designed using CAD software such as Onshape or SolidWorks and manufactured to accommodate all required components. A custom printed circuit board (PCB) will be developed to integrate the motor driver, inertial measurement unit (IMU), wheel encoders, power management circuitry, and wireless communication hardware. The system will utilize Wi-Fi connectivity to enable remote control and monitoring through a mobile device.

To further enhance the project's educational value, an additional balancing challenge will be incorporated into the control system. This feature will require the implementation of advanced feedback and control algorithms, providing an opportunity to explore concepts related to robot stability, sensor fusion, and dynamic system control.

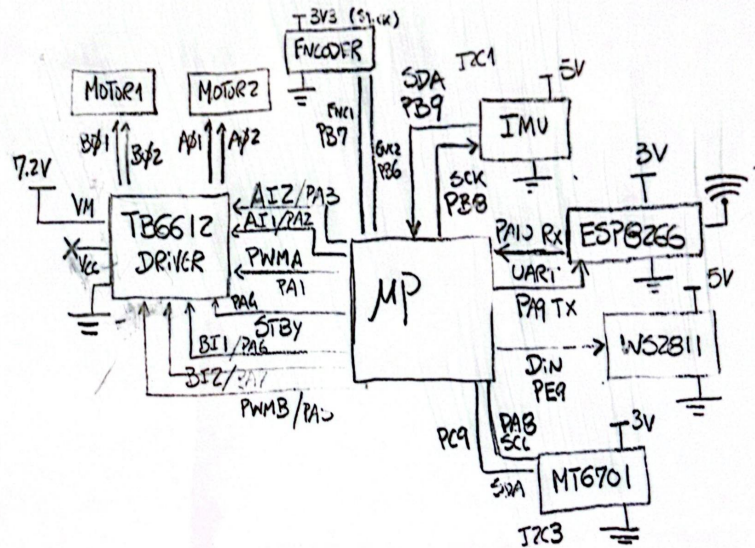


Design Stage and Block Diagram Development:

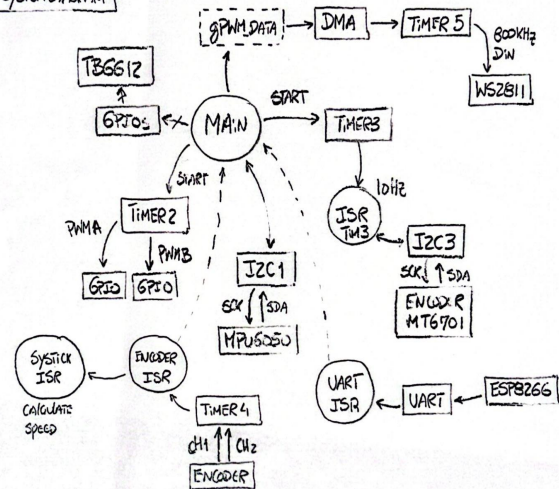
As with any embedded systems project, the design process began with the creation of block diagrams to clearly define the overall system architecture and the interactions between individual components. Developing these diagrams early in the design process helped establish the required hardware, communication interfaces, and software structure before implementation began.

The first block diagram represents the hardware architecture of the robot. At the center of the system is the microcontroller, which serves as the primary processing unit and coordinates communication between all subsystems. The microcontroller interfaces with a TB6612 motor driver that controls the two DC motors responsible for robot movement. Wheel encoder feedback is used to measure motor speed and position, enabling more accurate motion control. An IMU (Inertial Measurement Unit) communicates with the microcontroller through the I²C interface and provides acceleration and orientation data required for balancing and stability control. Wireless communication is achieved using an ESP8266 Wi-Fi module, allowing the robot to be monitored and controlled remotely from a smartphone. Additional peripherals, including an RGB LED and other communication devices, provide visual feedback and system status indications.

The second block diagram illustrates the software architecture of the system. The main program coordinates all robot operations and manages communication between hardware peripherals and software tasks. Multiple timers generate periodic interrupts used for motor control, encoder sampling, sensor updates, and system timing. Dedicated interrupt service routines (ISRs) process data from the encoders, IMU, and communication modules without blocking the main application. Pulse-width modulation (PWM) signals generated by the timers are used to control motor speed through the TB6612 driver. The Wi-Fi module communicates with the microcontroller through a UART interface, while the IMU and encoder devices communicate through I²C channels. This modular software structure improves reliability, simplifies debugging, and allows each subsystem to operate independently while remaining synchronized through the main control loop.



SYSTEM DIAGRAM



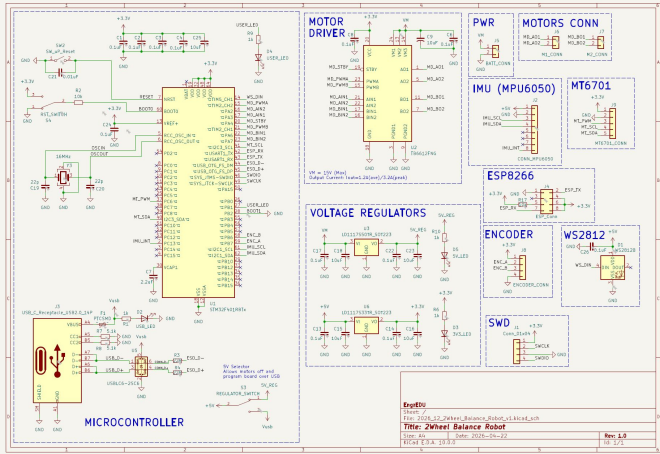
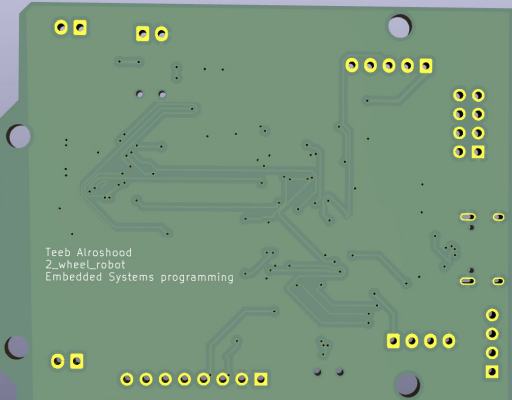
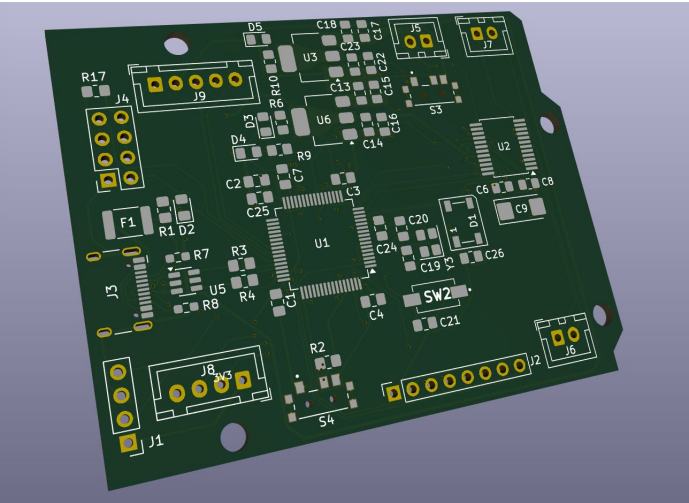
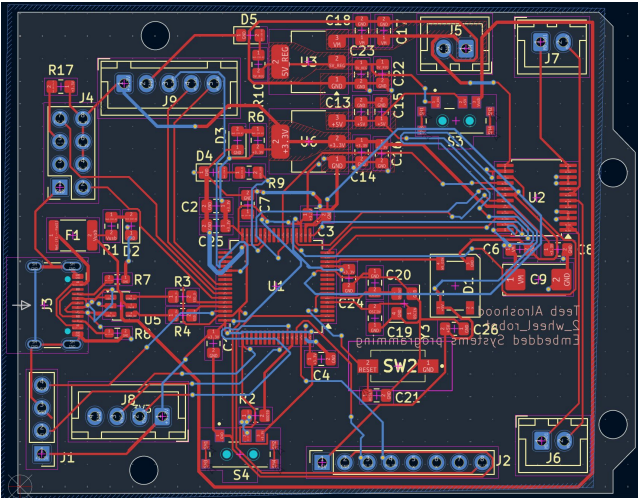
PCB Design and Development:

A custom PCB was designed to integrate the STM32 microcontroller, motor driver, IMU, magnetic encoders, RGB LED, and ESP8266 Wi-Fi module into a single compact platform. The schematic and PCB layout were developed to minimize wiring complexity and improve system reliability compared to a breadboard implementation.

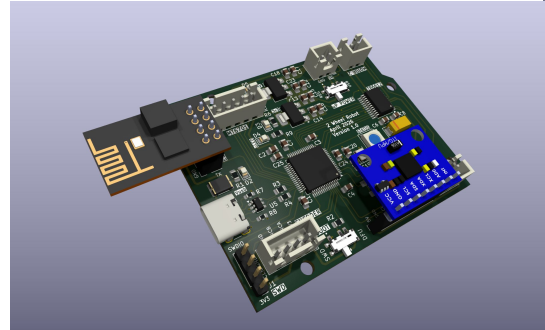
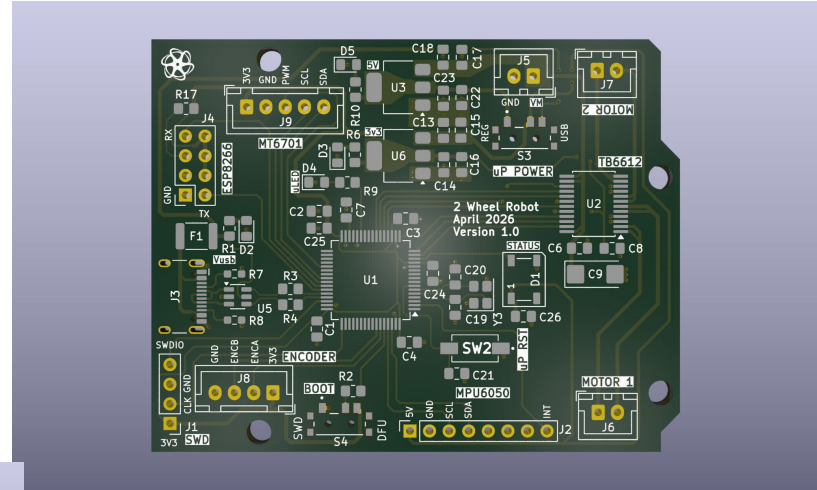
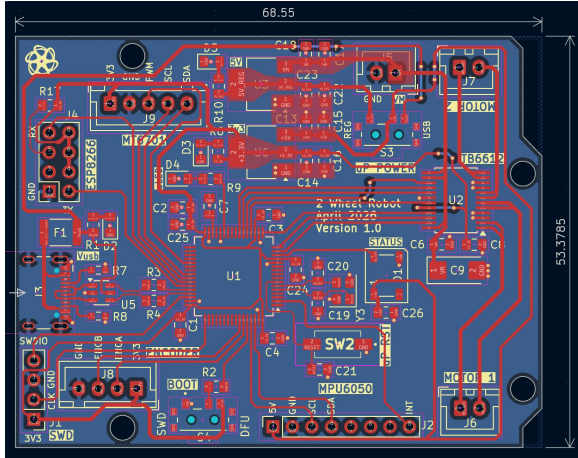
Although the custom PCB design was completed, unforeseen health related circumstances prevented full fabrication and testing within the project timeline. To ensure continued progress on the project, an instructor provided PCB with a similar hardware configuration was used for firmware development, peripheral validation, and system integration testing.

The first figure shows the custom PCB designed for this project, while the second figure shows the instructor provided PCB that was used during development. This approach allowed testing of the STM32 HAL drivers, UART communication with the ESP8266, I²C communication with the IMU and encoders, motor control functionality, and RGB LED operation while preserving the original PCB design work completed for the project.

Custom PCB Design:



Instructor PCB Design:



Prototype Stage:

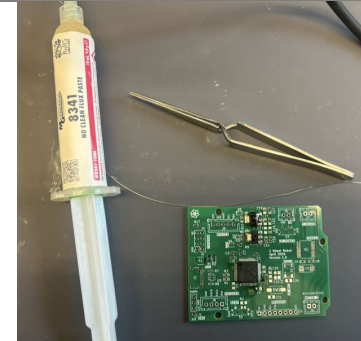
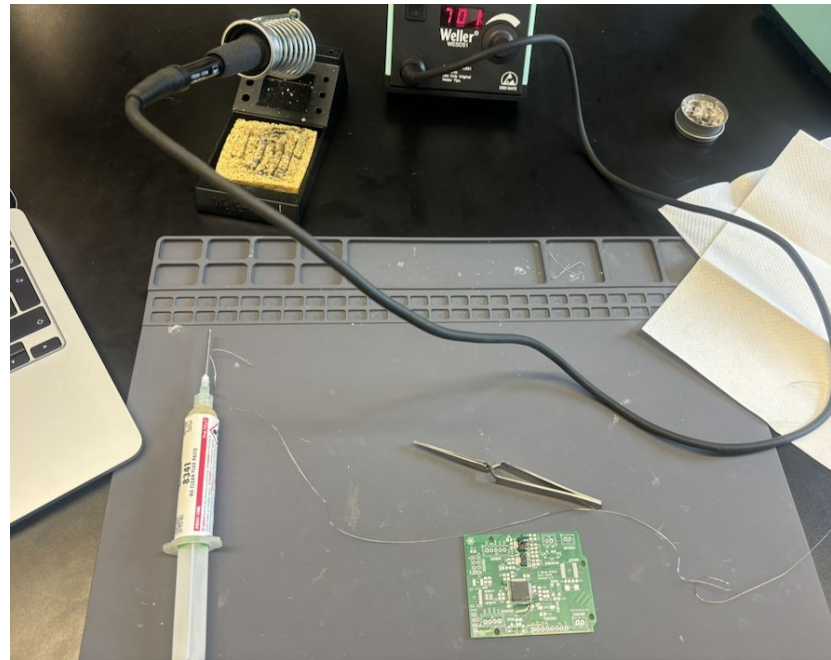
Custom PCB Assembly and Soldering

After verifying the functionality of the individual components using the STM32F407 development board and breadboard prototype, assembly of the custom PCB began. Surface Mount components were carefully soldered onto the board using solder paste, a soldering iron, and precision tweezers. This stage required attention to component orientation, solder joint quality, and proper placement of integrated circuits and passive components.

The first image shows the PCB during the assembly process alongside the soldering equipment used, including flux paste, tweezers, and the soldering station. These tools were essential for accurately placing and soldering the surface-mount devices onto the board.

The second image shows the partially assembled PCB after several key components had been soldered in place. At this stage, visual inspection was performed to identify solder bridges, cold joints, and component alignment issues before proceeding with power-up and functional testing.

Completing the PCB assembly represented a major milestone in the project, transitioning the system from a breadboard-based prototype to a compact and integrated hardware platform capable of supporting the robot's control, sensing, and wireless communication functions.

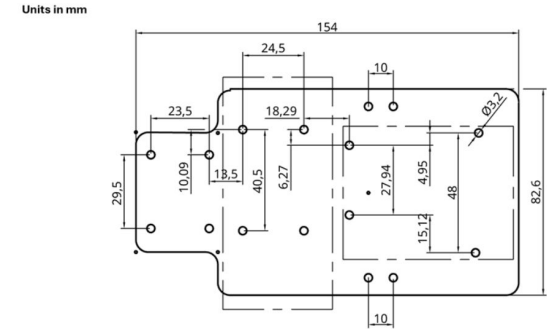
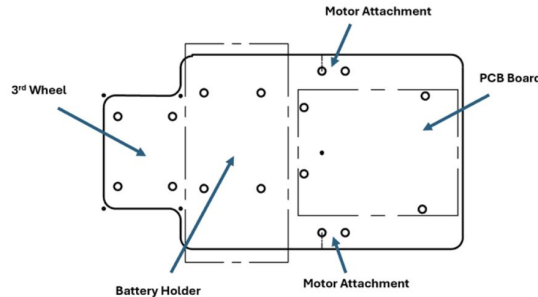
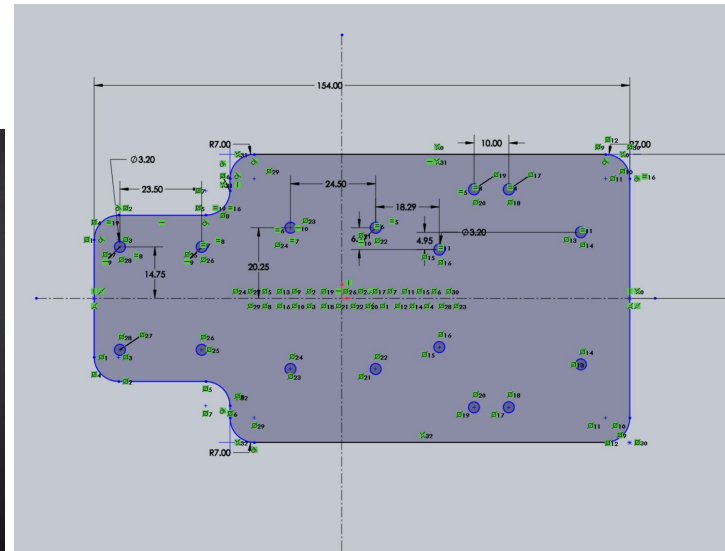


Chassis design and assembly:

The goal of the chassis design and acrylic mounting stage was to verify that all hardware components and the custom PCB functioned correctly before beginning development of the balancing control algorithm.

For the chassis design, I used the instructor's assembly as a reference and developed my own version using SolidWorks. The design was modified to accommodate the custom PCB, motors, battery, sensors, and mounting hardware while maintaining a compact and organized layout. The top assembly shown represents my SolidWorks design, while the bottom assembly is the instructor's original reference model.

Once assembled, the chassis was used to validate the embedded system hardware and software. Testing focused on verifying the STM32 HAL drivers for motor control, UART communication with the ESP8266 Wi-Fi module, I²C communication with the IMU and magnetic encoders, and operation of the RGB LED. Successfully validating these subsystems ensured that the hardware platform was fully functional before implementing the balancing control algorithm and closed-loop feedback system.

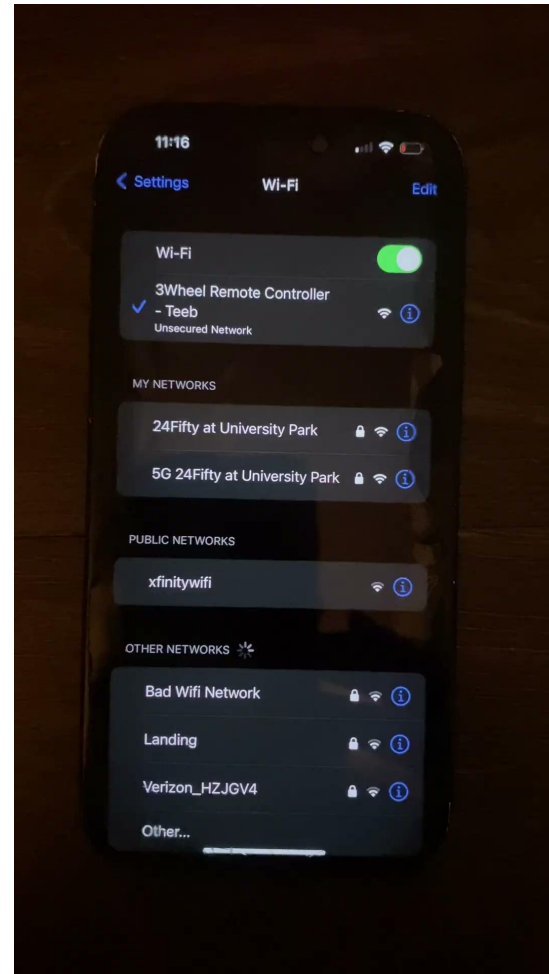


Wifi Module:

Wireless Control Demonstration

This video demonstrates the robot's wireless control system. The demonstration begins by connecting to the robot's dedicated Wi-Fi network through a smartphone. Once connected, a web-based remote control interface is opened in the phone's browser, allowing real-time communication with the robot.

The video then shows successful forward and backward motion commands being transmitted wirelessly, verifying proper operation of the Wi-Fi module, web interface, motor driver, and motor control system.



STM32 Motor Control Implementation

The robot was developed and programmed using STM32Cube IDE, which served as the primary development environment for writing, debugging, and testing the embedded software. STM32 programming was used to implement motor control, sensor communication, wireless control functionality, and overall system operation through the STM32 microcontroller.

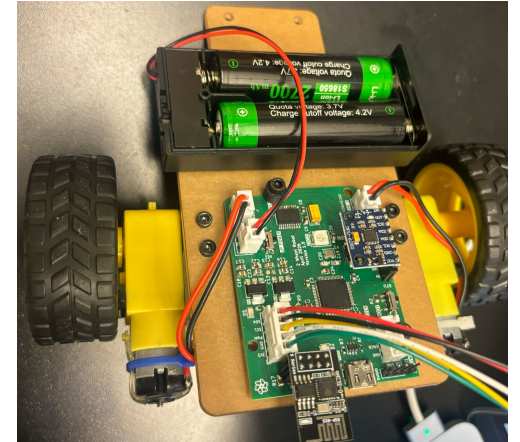
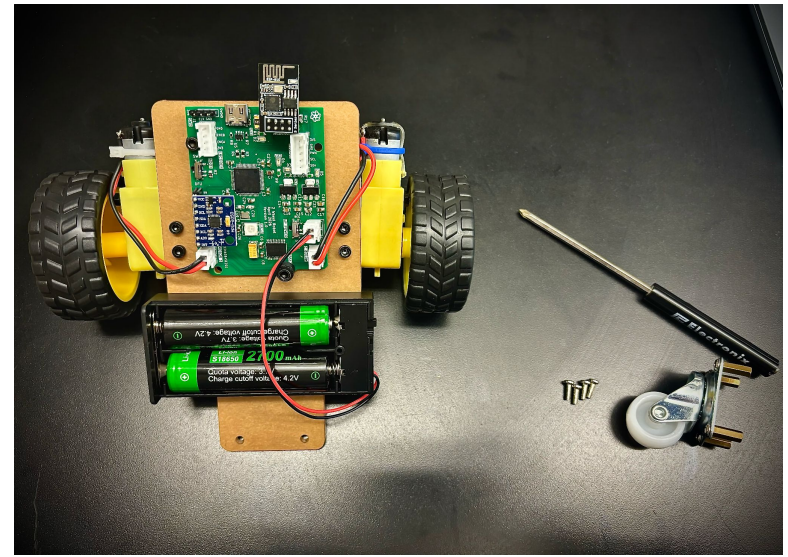


```
TB6612_Motor gMotorA = {  
    .htim = &htim2,  
    .Channel = TIM_CHANNEL_2,  
    .IN1_Port = AIN1_GPIO_Port,  
    .IN1_Pin = AIN1_Pin,  
    .IN2_Port = AIN2_GPIO_Port,  
    .IN2_Pin = AIN2_Pin,  
    .STBY_Port = STANDBY_GPIO_Port,  
    .STBY_Pin = STANDBY_Pin,  
    .Offset = 1  
};  
  
TB6612_Motor gMotorB = {  
    .htim = &htim2,  
    .Channel = TIM_CHANNEL_1,  
    .IN1_Port = BIN1_GPIO_Port,  
    .IN1_Pin = BIN1_Pin,  
    .IN2_Port = BIN2_GPIO_Port,  
    .IN2_Pin = BIN2_Pin,  
    .STBY_Port = STANDBY_GPIO_Port,  
    .STBY_Pin = STANDBY_Pin,  
    .Offset = 1  
};
```

An example of some of the code is this section which defines the two motors used for the two-wheel robot. Each motor is connected to the TB6612 motor driver using PWM timer channels and direction control pins. This allows the STM32 to control motor speed and direction.

Troubleshooting

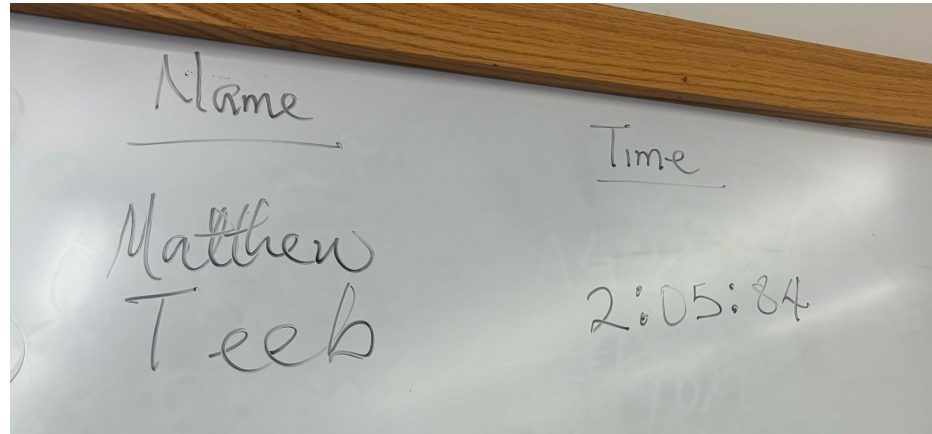
During the development and testing phase, the two-wheel self-balancing robot experienced stability issues and was unable to maintain balance consistently. Troubleshooting efforts focused on both hardware and software components. It was determined that the issue was likely related to the balancing code, specifically the tuning and integration of the control system with the IMU sensor data. Multiple adjustments and debugging sessions were performed to improve performance, however, further refinement of the software and control parameters is required to achieve stable balancing behavior.



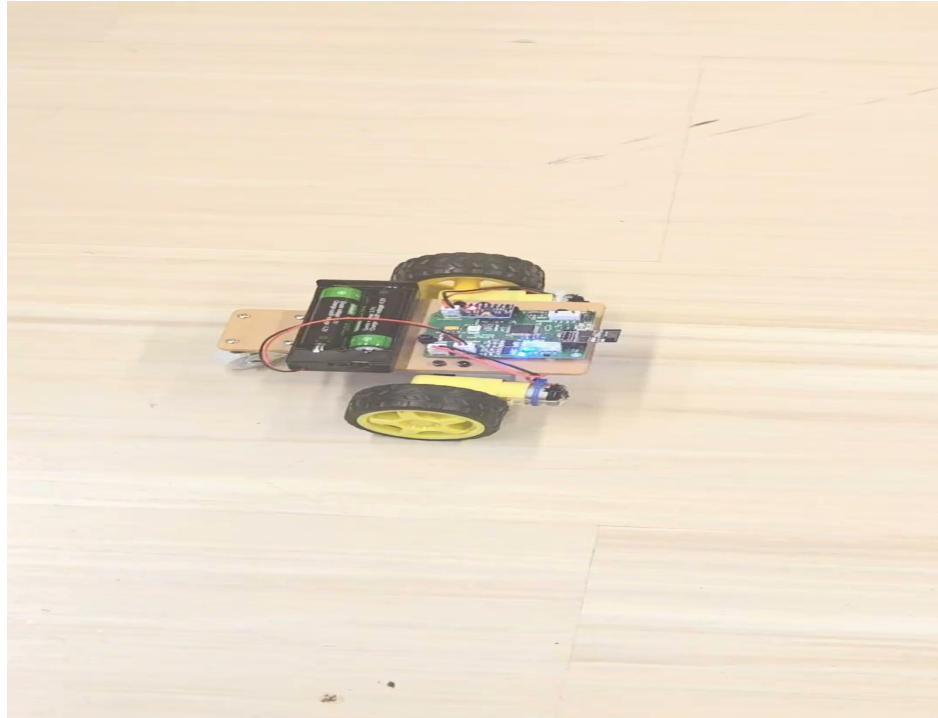
Surprise speed challenge



During class, a surprise speed challenge was introduced to test the performance of each team's three-wheel robot. The objective was to navigate through an obstacle course quickly and accurately while maintaining control and avoiding collisions. This challenge provided an opportunity to evaluate the robot's mechanical design, and software implementation under competitive conditions. It also highlighted the importance of efficient programming, when operating in a dynamic environment. Participating in the challenge allowed the team to compare its design against others, identify areas for improvement, and gain valuable insights.



Final Result (3 wheel):



Overall project achievements:

This project provided an opportunity to apply the concepts and skills I learned throughout the quarter in a hands on and engaging way. By designing, building, and programming a system using STM32IDE, I gained valuable experience integrating hardware and software components into a functional project. The experience strengthened my debugging skills, improved my ability to work within project deadlines, and enhanced my practical problem-solving and engineering thinking abilities.

Citations:

EngrEdu 2-Wheel Balance Robot website was used as a guide and reference during the design, assembly, and implementation phases of this project (EngrEdu, 2026).

<http://engredu.com/2026/05/01/2-wheel-balance-robot/>